

# Software License & Subscription Management System



Design & Implementation of a Relational Database for Centralized License Governance

|                         |                   |                                      |                        |
|-------------------------|-------------------|--------------------------------------|------------------------|
| STUDENT<br>Mihir Kalway | DEPARTMENT<br>SOT | PROFESSOR<br>Dr. Kiran Mayee Adavala | DATE<br>17th Sept 2025 |
|-------------------------|-------------------|--------------------------------------|------------------------|

# Licenses expire silently. Seats get over-allocated.



## THE PROBLEM

Organizations run many licensed products across departments with no centralized control — licenses expire unnoticed or seats are allocated beyond what was purchased.



## SCOPE

In scope — vendors, products, license types, purchases, allocation, subscriptions, renewals, compliance, reporting.

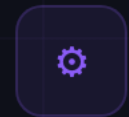
Out of scope — deployment, payment gateways, real-time telemetry.



## OBJECTIVES

Normalized 3NF schema with strong integrity constraints. Prevent over-allocation and duplicate keys. Enable expiry, utilization, and compliance reporting. Java/Python front-end for CRUD.

# User Roles & Functional Requirements



## IT Administrator

Maintains inventory, allocates and reclaims licenses



## Department Manager

Requests licenses for team members



## Procurement Manager

Records purchases and manages vendor renewals



## Compliance Auditor

Runs compliance checks and audit reports



## End User

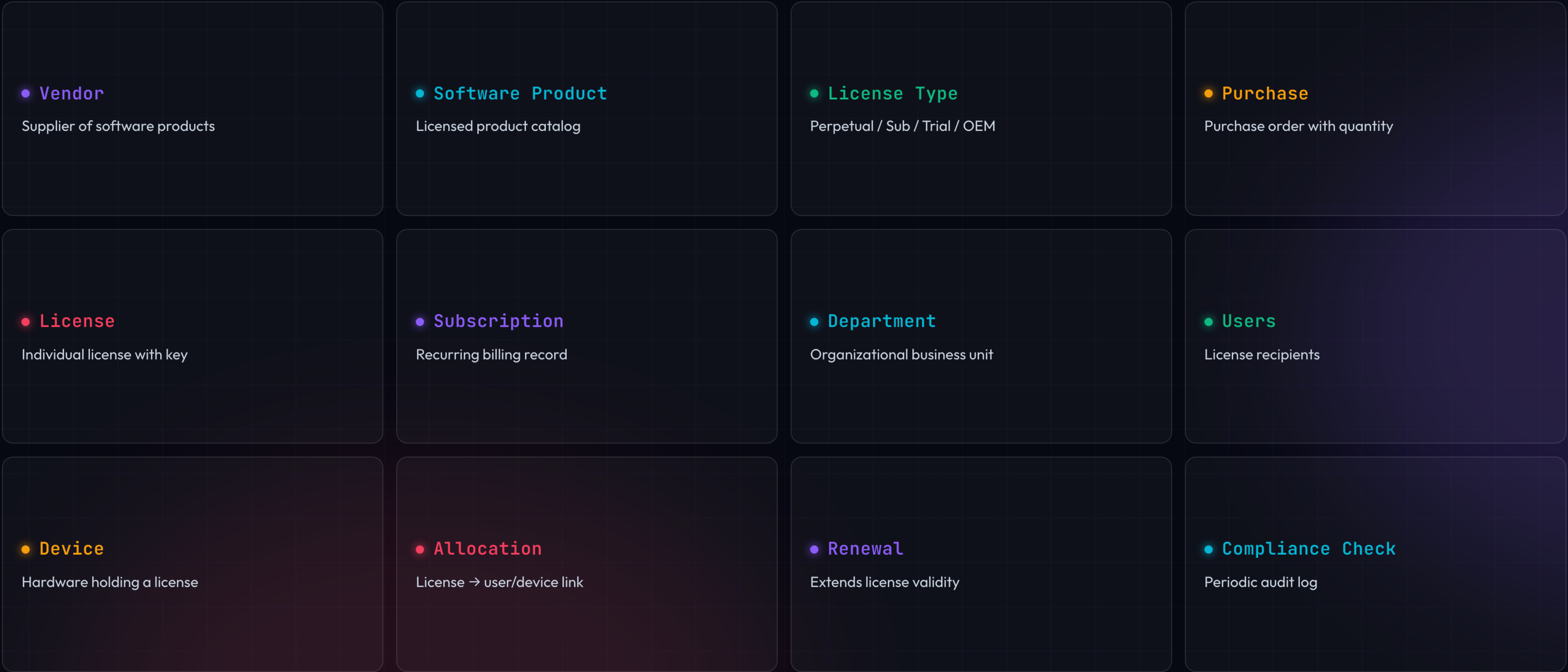
Uses allocated software on their device

## — FUNCTIONAL REQUIREMENTS

- ✓ Record vendors, products, purchases with unique license keys
- ✓ Allocate licenses within purchased seat quantity
- ✓ Track subscriptions and process renewals
- ✓ Reclaim licenses when users change roles
- ✓ Run compliance checks against active allocations
- ✓ Generate expiry, utilization, and compliance reports

# 12 core entities

forming the complete data model



# Structured Entity-Relationship Model

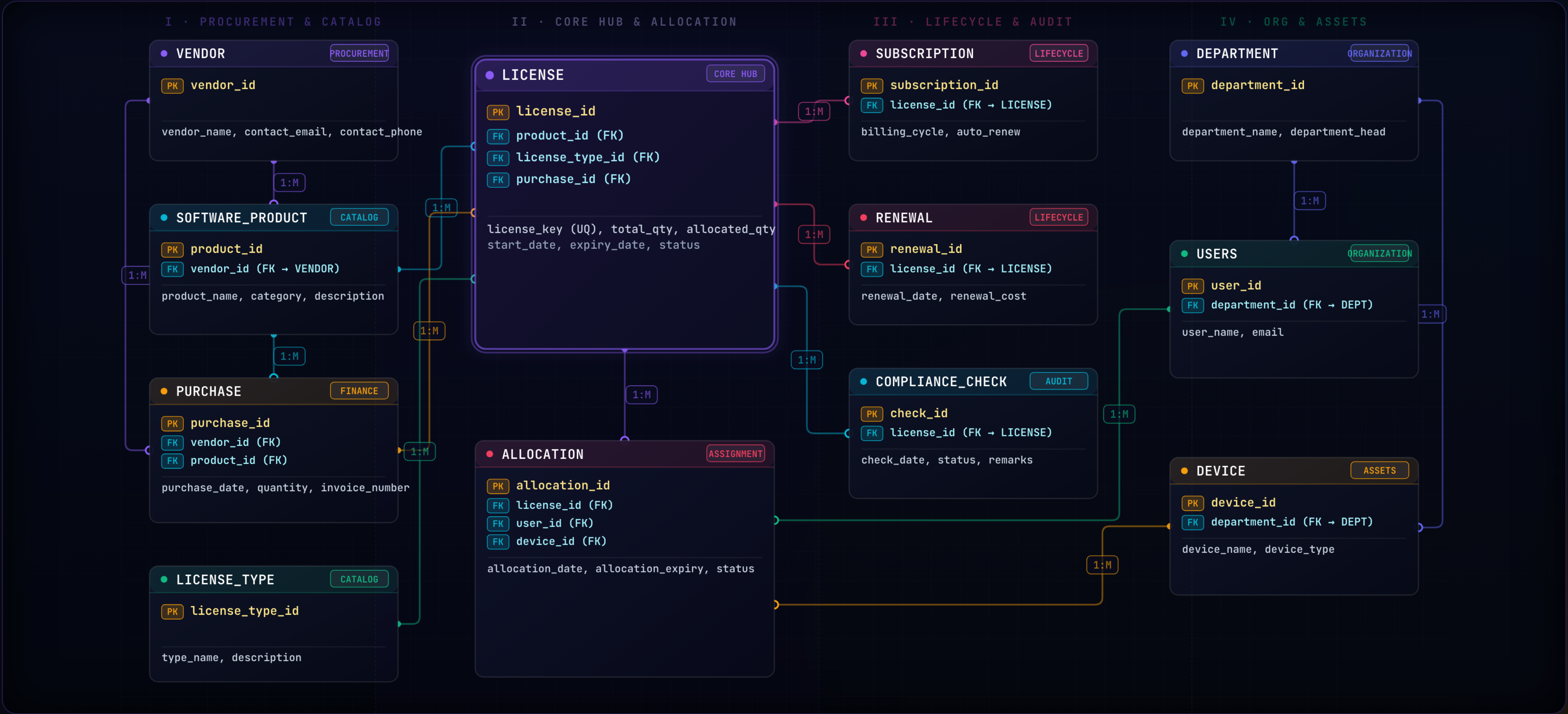
All Tables (12)

Procurement & Catalog (4)

Core Hub & Allocation (2)

Lifecycle & Audit (3)

Organization & Assets (3)



Architectural Legend · PK Primary Key · FK Foreign Key · 1:M One-to-Many Relational Cardinality

12 ENTITIES · 14 RELATIONSHIPS · 3NF

# Cardinalities & Meanings

| SOURCE                            | TARGET           | CARDINALITY | MEANING                                                   |
|-----------------------------------|------------------|-------------|-----------------------------------------------------------|
| Vendor                            | Product          | 1 : M       | One vendor supplies many software products                |
| Product / License Type / Purchase | License          | 1 : M       | Each entity generates multiple license records            |
| License                           | Subscription     | 1 : M opt   | Recurring billing for subscription-type licenses          |
| License                           | Allocation       | 1 : M       | One license (N seats) allocated to many users or devices  |
| License                           | Renewal          | 1 : M       | A license can be renewed multiple times over its lifetime |
| License                           | Compliance Check | 1 : M       | Compliance checks run repeatedly per license              |
| Department                        | Users / Device   | 1 : M       | A department owns many users and devices                  |

# Relational Schema, Part 1

| • VENDOR      |              |          |
|---------------|--------------|----------|
| vendor_id     | INT          | PK       |
| vendor_name   | VARCHAR(100) | NOT NULL |
| contact_email | VARCHAR(150) | UNIQUE   |
| contact_phone | VARCHAR(20)  |          |
| address       | TEXT         |          |

| • LICENSE_TYPE  |             |          |
|-----------------|-------------|----------|
| license_type_id | INT         | PK       |
| type_name       | VARCHAR(50) | NOT NULL |
| description     | TEXT        |          |

| • SOFTWARE_PRODUCT |              |             |
|--------------------|--------------|-------------|
| product_id         | INT          | PK          |
| product_name       | VARCHAR(150) | NOT NULL    |
| vendor_id          | INT          | FK → VENDOR |
| category           | VARCHAR(50)  |             |
| description        | TEXT         |             |

| • PURCHASE     |             |              |
|----------------|-------------|--------------|
| purchase_id    | INT         | PK           |
| vendor_id      | INT         | FK → VENDOR  |
| product_id     | INT         | FK → PRODUCT |
| purchase_date  | DATE        | NOT NULL     |
| quantity       | INT         | CHECK > 0    |
| invoice_number | VARCHAR(50) | UNIQUE       |

PK Primary key    FK Foreign key    UNIQUE Unique constraint

# Relational Schema, Part 2

|                 |             |               |
|-----------------|-------------|---------------|
| ● SUBSCRIPTION  |             |               |
| subscription_id | INT         | PK            |
| license_id      | INT         | FK → LICENSE  |
| billing_cycle   | VARCHAR(20) |               |
| auto_renew      | BOOLEAN     | DEFAULT FALSE |

|                 |              |                 |
|-----------------|--------------|-----------------|
| ● DEPARTMENT    |              |                 |
| department_id   | INT          | PK              |
| department_name | VARCHAR(100) | UNIQUE NOT NULL |
| department_head | VARCHAR(100) |                 |

|               |              |                 |
|---------------|--------------|-----------------|
| ● USERS       |              |                 |
| user_id       | INT          | PK              |
| user_name     | VARCHAR(100) | NOT NULL        |
| email         | VARCHAR(150) | UNIQUE NOT NULL |
| department_id | INT          | FK → DEPT       |

|               |              |           |
|---------------|--------------|-----------|
| ● DEVICE      |              |           |
| device_id     | INT          | PK        |
| device_name   | VARCHAR(100) | NOT NULL  |
| device_type   | VARCHAR(50)  |           |
| department_id | INT          | FK → DEPT |

|                   |             |                    |
|-------------------|-------------|--------------------|
| ● ALLOCATION      |             |                    |
| allocation_id     | INT         | PK                 |
| license_id        | INT         | FK → LICENSE       |
| user_id           | INT         | FK nullable        |
| allocation_expiry | DATE        | CHECK ≥ alloc_date |
| status            | VARCHAR(20) | DEFAULT 'Active'   |

|              |         |              |
|--------------|---------|--------------|
| ● RENEWAL    |         |              |
| renewal_id   | INT     | PK           |
| license_id   | INT     | FK → LICENSE |
| renewal_date | DATE    | NOT NULL     |
| renewal_cost | DECIMAL | CHECK > 0    |

|                    |             |              |
|--------------------|-------------|--------------|
| ● COMPLIANCE_CHECK |             |              |
| check_id           | INT         | PK           |
| license_id         | INT         | FK → LICENSE |
| check_date         | DATE        | NOT NULL     |
| status             | VARCHAR(20) |              |
| remarks            | TEXT        |              |

# The database enforces these rules

Not the application layer — the schema itself.

|                                 |                                   |
|---------------------------------|-----------------------------------|
| ● Unique License Keys           | UNIQUE ON license.license_key     |
| ● Allocation Within Seat Count  | CHECK (allocated_qty ≤ total_qty) |
| ● Valid Allocation Period       | CHECK (expiry ≥ allocated_date)   |
| ● No Expired License Assignment | WHERE status = 'Active'           |
| ● Positive Renewal Cost         | CHECK (renewal_cost > 0)          |
| ● Referential Integrity         | FOREIGN KEY ... REFERENCES        |

↑ Click any rule to expand details

# DDL Schema & Relational Constraints

Production MySQL 8.0 / PostgreSQL tables with referential integrity, check constraints, and performance indexes.

presentation2/ddl.sql

- Core Licenses & Products
- Allocations & Devices
- Departments & Users
- Renewals & Audit Logs

01\_core\_schema.sqlCopy SQL

```
-- CORE LICENSES & SOFTWARE PRODUCTS
CREATE TABLE licenses (
  license_id      INT AUTO_INCREMENT PRIMARY KEY,
  license_key     VARCHAR(128) NOT NULL UNIQUE,
  product_id     INT NOT NULL,
  license_type_id INT NOT NULL,
  purchase_id     INT,
  total_seats     INT NOT NULL DEFAULT 1,
  allocated_seats INT NOT NULL DEFAULT 0,
  purchase_date   DATE NOT NULL,
  expiry_date     DATE NOT NULL,
  unit_cost       DECIMAL(12, 2) NOT NULL DEFAULT 0.00,
  total_cost      DECIMAL(12, 2) NOT NULL DEFAULT 0.00,
  billing_cycle   ENUM('MONTHLY', 'ANNUALLY', 'ONE_TIME') DEFAULT 'ANNUALLY',
  auto_renew      BOOLEAN DEFAULT FALSE,
  status          ENUM('ACTIVE', 'EXPIRED', 'SUSPENDED', 'PENDING_RENEWAL') DEFAULT 'ACTIVE',
  -- Relational Integrity Constraints
  CONSTRAINT fk_license_product FOREIGN KEY (product_id)
    REFERENCES software_products(product_id) ON DELETE CASCADE,
  CONSTRAINT chk_allocated_seats CHECK (allocated_seats ≤ total_seats),
  CONSTRAINT chk_license_dates  CHECK (expiry_date ≥ purchase_date),
  CONSTRAINT chk_unit_cost      CHECK (unit_cost ≥ 0.00)
);

CREATE INDEX idx_licenses_prod_status ON licenses(product_id, status);
CREATE INDEX idx_licenses_expiry     ON licenses(expiry_date);
```

## ENFORCED DATABASE INVARIANTS

Seat Invariant

CHECK (allocated\_seats ≤ total\_seats)

Database engine rejects any transaction that causes over-licensing.

Chronological Bounds

CHECK (expiry\_date ≥ purchase\_date)

Guarantees contract validity dates cannot be inverted.

Cascading Deletion

ON DELETE CASCADE

Removing a software product cleanly purges associated licenses without orphan rows.

⚡ Fully Executable Master Script Ready

Contains 12 normalized tables with MySQL & PostgreSQL compatibility.

# Transactional Seat Lifecycle & Operations

ACID-compliant transactions ensuring seat counts never drift, corrupt, or deadlock during concurrent IT provisioning.

presentation2/dml.sql

- Seat Allocation Transaction
- Seat Reclamation / Offboarding
- Annual Contract Renewal
- Enterprise Seed Data Population

tx\_allocate\_seat.sqlCopy SQL

```
-- TRANSACTION 1: ATOMIC SEAT PROVISIONING
START TRANSACTION;

-- Step 1: Row-lock the license record to eliminate concurrent race conditions
SELECT license_id, total_seats, allocated_seats, status
FROM licenses
WHERE license_id = 1
FOR UPDATE;

-- Step 2: Capacity check validated (allocated_seats < total_seats)
-- Insert new allocation record mapping employee & workstation
INSERT INTO license_allocations (
  allocation_code, license_id, user_id, device_id,
  allocated_date, allocation_expiry, status, allocated_by
) VALUES (
  CONCAT('ALC-', DATE_FORMAT(NOW(), '%Y%m%d'), '-', LPAD(FLOOR(RAND() * 9999), 4, '0')),
  1, 2, 2,
  CURRENT_DATE(), DATE_ADD(CURRENT_DATE(), INTERVAL 1 YEAR), 'ACTIVE', 'IT Admin'
);

-- Step 3: Increment allocated seats atomically
UPDATE licenses
SET allocated_seats = allocated_seats + 1
WHERE license_id = 1;

COMMIT;
```

ACID TRANSACTION PROTOCOL

- 01

**Row-Level Pessimistic Lock**  
SELECT FOR UPDATE prevents simultaneous transactions from oversubscribing the last remaining seat.
- 02

**Junction Record Creation**  
Creates verified license\_allocations mapping with immutable audit code and timestamps.
- 03

**Atomic Counter Synchronization**  
Increments allocated\_seats in parent table; rolls back on any violation.
- Zero Phantom Allocations Guaranteed**  
Database-level row locks ensure concurrent IT admins cannot over-allocate seats.

# Enterprise Reporting & Analytical Queries

Real-world SQL queries powering expiration alerts, capacity warnings, department chargebacks, and compliance audits.

presentation2/queries.sql

- 🔥 30-Day Expiration Alert
- 📊 ⚡ Seat Saturation (≥ 90%)
- 💰 🏢 Department Software Spend
- 📊 🏢 Vendor Spend Concentration
- 🛡️ 📋 Compliance & Audit Risks

analytical\_query.sql Copy SQL

```
-- QUERY 1: 30-DAY LICENSE EXPIRATION ALERT & PIPELINE
SELECT
  l.license_id,
  v.vendor_name,
  sp.product_name,
  l.total_seats,
  l.allocated_seats,
  l.expiry_date,
  DATEDIFF(l.expiry_date, CURRENT_DATE()) AS days_left,
  l.total_cost AS renewal_cost,
  CASE
    WHEN DATEDIFF(l.expiry_date, CURRENT_DATE()) ≤ 7 THEN 'CRITICAL'
    WHEN DATEDIFF(l.expiry_date, CURRENT_DATE()) ≤ 15 THEN 'URGENT'
    ELSE 'UPCOMING'
  END AS urgency_level
FROM licenses l
JOIN software_products sp ON l.product_id = sp.product_id
JOIN vendors v ON sp.vendor_id = v.vendor_id
WHERE l.status IN ('ACTIVE', 'PENDING_RENEWAL')
  AND l.expiry_date BETWEEN CURRENT_DATE() AND DATE_ADD(CURRENT_DATE(), INTERVAL 30 DAY)
ORDER BY l.expiry_date ASC;
```

RENEWAL PIPELINE  
**\$46,525 at risk** [Live SQL View](#)

SIMULATED QUERY EXECUTION RESULT

| PRODUCT                | VENDOR     | SEATS   | EXPIRY DATE | DAYS LEFT | RENEWAL COST | URGENCY  |
|------------------------|------------|---------|-------------|-----------|--------------|----------|
| Adobe Creative Cloud   | Adobe Inc. | 24 / 25 | 2026-02-01  | 14 days   | \$21,575     | URGENT   |
| IntelliJ IDEA Ultimate | JetBrains  | 48 / 50 | 2026-02-15  | 28 days   | \$24,950     | UPCOMING |

# Tools, Stack & Timeline

● TECHNOLOGY STACK

● DATABASE

MySQL / PostgreSQL

● BACKEND

Java (JDBC + Swing) or Python (Tkinter/Flask)

● DIAGRAMS

draw.io — ER diagram export

● VERSION CONTROL

GitHub — feature-branch workflow

| ● DELIVERY TIMELINE |         |                                                               |
|---------------------|---------|---------------------------------------------------------------|
| ✓ Review 1          | Week 7  | Problem definition, ER diagram, initial schema                |
| ✓ Review 2          | Week 11 | 3NF schema, DDL, sample data, SQL queries & views (Delivered) |
| ○ Review 3          | Week 16 | Working application, final demo, report, viva                 |

Thank you.

QUESTIONS WELCOME